

High-Throughput ECC2K-130 Walks on Blackwell GPUs

Packed Binary-Field Arithmetic and Cooperative Batch Inversion

Adam Buran

September 21, 2026

Research preprint draft

Abstract

Efficient elliptic-curve collision-search implementations require a joint treatment of finite-field arithmetic, walk semantics, and the GPU memory hierarchy. We describe an implementation for the public ECC2K-130 parameters over $\mathbb{F}_{2^{131}}$ using packed polynomial coordinates, native carryless multiplication, an eight-branch table walk, and cooperative inversion across groups of four warps. The last arithmetic refinement reduces a polynomial square in compressed even and odd coefficient streams before interleaving the result. Together with a rotated four-warp inversion tree, explicit non-aliasing buffers, compact metadata, and a 640-thread block geometry, the accepted configuration reaches a median of **22.100934 billion scheduled scalar-slot updates per second** on one NVIDIA RTX PRO 6000 Blackwell Server Edition. Five confirmation runs each execute 504,658,657,280 updates; their range is 22.089525–22.166455 billion updates/s. The median improvement over an alternating, same-device baseline is **11.32%**; every candidate sample exceeds every baseline sample. Screening runs attribute most of that gain to the inversion tree at the wider geometry and three to four percentage points to the square, but the components were not isolated in a controlled ablation. A separate distinguished-point collection run reaches **21.548499 billion updates/s** and produces 12,387 records with zero reported drops. Independent arithmetic probes, 300 reference trail replays, equal complete report corpora, and bidirectional checkpoint continuation support the implementation evidence. We distinguish these results from effective collision-search throughput: history-dependent cycle avoidance, exceptional affine inputs, and long-run collision statistics remain outside the accepted performance claim. The work establishes a reproducible GPU implementation result, without asserting a completed ECC2K-130 solution or a new asymptotic discrete-logarithm algorithm.

Keywords: ECC2K-130; Koblitz curves; GPU cryptanalysis; binary fields; batch inversion; carryless multiplication; reproducible benchmarking.

1 Introduction

The ECC2K-130 challenge provides a concrete setting in which to study the cost of elliptic-curve computation over a binary field. Its arithmetic is especially suitable for low-level optimization: field addition is XOR, the Frobenius endomorphism is linear, and many independent walks can share inversions. These features also create a difficult implementation tradeoff. A larger batch saves inversions but increases the working set; more parallel work can hide latency but consumes registers and shared memory; a cheaper iteration function can change the statistical behavior on which collision search depends.

This paper studies the current accepted high-throughput research configuration in the project’s archived ECC2K-130 implementation, identified as `warps4_poly_compact640`. The

source and measurement receipt are fixed by content hashes. The result is a complete walk-loop measurement on one GPU, including selection and coordinate updates, rather than an isolated multiplication benchmark. Its central finding is that arithmetic specialization and scheduling must be designed together. Cooperative inversion alone was not sufficient in earlier experiments; the accepted combination uses a rotated product tree, compact live metadata, restricted pointers, and a specialized polynomial square.

The contributions are:

1. A precise description of the eight-branch, automorphism-aware table walk and of the history that its short-cycle rule adds to the state.
2. A two-level inversion schedule whose multiplication budget is $5 + 5/(BW)$ per ordinary affine update, with $B = 16$ slots per thread and $W = 4$ collaborating warps.
3. An exact even/odd-stream square-and-reduce map for the internal degree-131 polynomial representation, accompanied by a reproducible basis-vector identity check.
4. An evidence-backed evaluation of a 22.100934-billion-update/s configuration, including all five confirmation samples, a configuration-by-configuration comparison with the baseline, the screening sequence that led to it, resource usage, collection performance, and validation limits.
5. First-order estimates for the walk’s open questions: the unreachable part of the selector domain, the probability of an exceptional addition, merge coalescence under the history rule, and the frequency of six-step fruitless cycles.

The measured implementation is the strongest accepted configuration in the evidence set examined here. We use *state of the art* only as a research objective and literature context: a global fastest-solver claim would additionally require a current, comparable survey and validated collision efficiency for this particular walk. The archived result does not supply either a completed challenge solution or that full comparison.

2 Background and comparison scope

Pollard’s rho method provides the classical low-memory approach to generic discrete logarithms [10]. Distinguished-point methods permit many processors to contribute trails to a common collision search [11]. In that setting, iteration speed is only one component of performance: trail coalescence, fruitless cycles, endpoint production, replay, and collision usefulness all affect the work needed for a successful computation.

The ECC2K-130 work of Bailey et al. exploits Frobenius and negation symmetries and analyzes a specific iteration function [1]. Bernstein et al. subsequently report more than 63 million iterations/s on a GTX 295 graphics card, using bitsliced arithmetic and carefully managed GPU storage [2]. Bos et al. report 25.57 million iterations/s across the six usable SPUs of a PlayStation 3 [5]. These are valuable historical reference points, but the hardware, arithmetic representation, and iteration function differ from the present table-walk implementation.

Batch inversion and normal-basis exponentiation are established techniques [8, 7]. Likewise, the difficulty of exploiting automorphisms without introducing fruitless cycles is a known issue [4]. Our contribution is a concrete implementation and its measured combination of optimizations. We do not claim to introduce Pollard rho, the use of Frobenius orbits, table walks in general, or the batch-inversion principle.

Work	Hardware / scope	Rate	Unit
Bos et al. [5]	Cell, six SPUs; historical walk	25.57	million/s
Bernstein et al. [2]	GTX 295 card; historical walk	> 63	million/s
Local control	One RTX PRO 6000; table walk	19.852741	billion/s
This configuration	Same GPU; same table walk	22.100934	billion/s
Collection mode	Same candidate; DP weight 34	21.548499	billion/s

Table 1: Contextual throughput comparison. The first two rows are published historical iteration rates. The last three use the scheduled-slot metric in Section 6. Only the two local benchmark rows constitute a direct implementation comparison; the collection row is a single separate run.

3 Curve, representations, and walk semantics

3.1 Curve and field

The implemented curve is

$$E : y^2 + xy = x^3 + 1 \quad \text{over } \mathbb{F}_{2^{131}}, \quad (1)$$

with the prime-order subgroup parameter recorded in the generated source as

$$\ell = 680564733841876926932320129493409985129. \quad (2)$$

For an affine point $R = (x, y)$, negation is $-R = (x, y + x)$ and Frobenius is $\sigma(R) = (x^2, y^2)$. The ordinary automorphism orbit is $\{\pm\sigma^k(R) : 0 \leq k < 131\}$, of size at most 262. Orbit arithmetic and exceptional orbit sizes must be distinguished from a heuristic uniform random map on $\ell/262$ objects.

The code uses a permuted type-II optimal normal basis indexed by $e \in \{1, \dots, 131\}$, with $263 = 2 \cdot 131 + 1$ and the index operation

$$\text{fold}(u) = \min(u \bmod 263, 263 - (u \bmod 263)). \quad (3)$$

Squaring maps index e to $\text{fold}(2e)$. A normal-basis coordinate vector is useful for Hamming weight and walk selection. Persistent coordinates and most multiplication intermediates instead use an internal polynomial basis; conversion is an exact generated linear map.

The internal reduction polynomial is represented by the bit string

$$F = 0xD1D0D000D0000000D00000000000000D. \quad (4)$$

Equivalently, with the bit in position i denoting the coefficient of z^i ,

$$F(z) = z^{124} + (1 + z^2 + z^3)(1 + z^{64} + z^{96} + z^{112} + z^{120} + z^{128}). \quad (5)$$

This internal representation is distinct from the sparse polynomial $z^{131} + z^{13} + z^2 + z + 1$ also listed by the parameter generator, and the choice is structural. Let ζ be a primitive 263rd root of unity and $\beta = \zeta + \zeta^{-1}$, so that the normal-basis elements are $\beta_e = \zeta^e + \zeta^{-e}$. The minimal polynomial of β over $\mathbb{F}_2$ is produced by the recurrence

$$f_0 = 1, \quad f_1 = z + 1, \quad f_{j+1} = zf_j + f_{j-1}, \quad (6)$$

and a direct computation confirms $f_{131} = F$ exactly. The internal polynomial basis is therefore $1, \beta, \dots, \beta^{130}$: the same element β generates both representations. Since

$$\beta^j = \sum_{0 \leq i < j/2} \binom{j}{i} \beta_{j-2i} \quad \text{in characteristic two,} \quad (7)$$

the change of basis is a fixed and highly structured linear map, closely related to the type-II optimal polynomial bases of Bernstein and Lange [3]. This is what makes it affordable to keep persistent state in polynomial form while still evaluating the normal-basis weight and selector on every step.

The price is a denser modulus: F has 19 terms where the pentanomial has five. The factored form above limits that cost, because reduction applies only the three-term factor $1+z^2+z^3$ at six byte-aligned offsets, together with the single term z^{124} . With the pentanomial, the selector would instead need an unstructured 131×131 basis change on every step. That alternative was not measured, so the tradeoff is offered as the design rationale and not as a measured optimum. The square circuit below is tied to Equation (4); substituting another representation without changing the linear maps would be incorrect.

3.2 Eight-branch table selection

Let P, Q be the public challenge points. The reference implementation deterministically constructs $H = 8$ table points $T_h = [a_h]P + [b_h]Q$, along with all $\sigma^k(T_h)$. The coefficients are fixed by a public branch-index seed and modular reduction; they are part of the walk identity. The device reads the table coordinates in polynomial representation.

Write the normal-basis x coordinate as (x_e) . Define L by

$$L(\text{fold}(2^t)) = t, \quad 0 \leq t < 131. \quad (8)$$

For $w = \sum_e x_e$ with $w \not\equiv 0 \pmod{131}$, the raw selection is

$$h(R) = \lfloor w/2 \rfloor \bmod 8, \quad (9)$$

$$k(R) = w^{-1} \sum_e L(e)x_e \pmod{131}, \quad (10)$$

$$p(R) = \arg \max_{e:x_e=1} ((L(e) - k(R)) \bmod 131), \quad (11)$$

$$\epsilon(R) = y_{p(R)}. \quad (12)$$

Before the cycle correction, the update is

$$f(R) = R + (-1)^{\epsilon(R)} \sigma^{k(R)}(T_{h(R)}). \quad (13)$$

Only the selected normal-basis coordinate of y is needed: a precomputed linear row evaluates it from polynomial y , avoiding a full conversion. The x conversion is retained for the selector and distinguished-point test.

Proposition 1 (Raw-selector equivariance). *On the domain where the selector and affine update are defined, the raw map in Equation (13) satisfies $f(\sigma R) = \sigma f(R)$ and $f(-R) = -f(R)$.*

Proof. Squaring permutes the support and adds one to every $L(e)$ modulo 131. The weight is unchanged, so k increases by one and the cyclic ordering defining the pivot is preserved. The selected y bit is carried along with its pivot, making ϵ Frobenius-invariant. Under negation x does not change, while y becomes $y + x$. The pivot lies in the support of x , hence its y bit flips. Applying these transformations to the chosen addend gives the two identities. $\square$

The statement has an explicit domain: a zero coordinate has no pivot, and weights divisible by 131 do not admit the displayed inverse. Inside the prime-order subgroup this part of the domain is never left. Every subgroup point is a double, so $\text{Tr}(x) = \text{Tr}(a_2) = 0$ for this curve, whose x^2 coefficient a_2 is zero. Each element of the normal basis has trace one, hence $\text{Tr}(x) \equiv w \pmod{2}$ and the weight w is always even. This is also why the branch index uses

$\lfloor w/2 \rfloor$. Since $0 \leq w \leq 131$ and 131 is odd, $w \equiv 0 \pmod{131}$ forces $w = 0$, that is $x = 0$. The only affine point with $x = 0$ is $(0, 1)$, which has order two and does not lie in the subgroup of order ℓ . The selector is therefore defined at every point a trail can reach. The remaining exceptional case is the affine addition itself, which is treated in Section 8.2.

3.3 History-dependent cycle avoidance

The actual implementation augments the point with the three most recent step tags. A tag is (h, k, ϵ) , packed into a 16-bit value. If the proposed tag is the sign-opposite of the preceding tag, it would immediately undo that addition. The code also rejects the four-step pattern where the new tag cancels the second previous tag and the most recent tag cancels the third previous tag. It increments h modulo eight until the test clears, then updates history.

At most two branch values can be prohibited by these two equality tests for a fixed (k, ϵ) , so the branch scan terminates for eight branches. The rule removes those particular algebraic cancellation patterns. It neither enumerates all possible cycles nor proves a random-map collision constant.

Most significantly, the resulting transition is a function of *point and history*. Two trails at the same point can select different successors when their histories differ. Equivariance extends to consistently transformed histories, but it does not by itself prove that projected point collisions coalesce as in a point-only rho walk.

A first-order estimate shows that the effect on coalescence is small. Write $r = 2 \cdot 8 \cdot 131 = 2,096$ for the number of signed addends, and model the raw tag as uniform and independent of the history. When two trails meet at a point, both propose the same raw tag t . The correction changes the successor of a trail only if t cancels that trail’s most recent tag, which has probability about $1/r$, or completes the four-step pattern, which has probability about $1/r^2$. The two trails therefore take different successors at the meeting point with probability about $2/r \approx 9.5 \times 10^{-4}$. After one shared step their most recent tags agree, so the two-step test agrees; after three shared steps the three stored tags agree and the trails are identical from then on. In this model roughly 0.1% of merges fail to coalesce, which is negligible against the other constants of the search.

Longer fruitless cycles are not negligible by the same argument. A six-step cycle a, b, c followed by $-a, -b, -c$ in some order survives the two rules for three of the six orders, and a trail that enters it repeats it indefinitely. In the same model the probability per step is about $3/r^3 \approx 3.3 \times 10^{-10}$. A DP34 trail has mean length $2^{25.27} \approx 4.0 \times 10^7$ steps (Section 3.4), so on the order of 1% of trails would enter a six-step cycle before reaching a distinguished point; cycles of length eight and more are rarer by further factors of r . The client has a guard for this case: `-max-iters` restarts any walk that exceeds a step limit, checked every 4,096 steps. The guard is disabled by default and was not enabled in the measured commands. If the estimate is right, the limit matters, because a trapped trail consumes the whole limit: at the customary limit of 20 mean trail lengths [11], a 1% trapped fraction would cost between a sixth and a fifth of all work. These figures are heuristic and come from the uniform model above, following the analysis style of Bernstein, Lange, and Schwabe [4]. They have not been measured for this selector, whose branch index is not exactly uniform. We therefore separate arithmetic equivalence and finite corpus equality from a full collision-efficiency claim.

3.4 Distinguished points and replay

The collection predicate is $\text{HW}(x_{\text{NB}}) \leq 34$. Thus “DP34” denotes a weight threshold; it does not denote a probability of 2^{-34} . Because the weight is always even, the probability that a

uniform subgroup point is distinguished is

$$\theta_{34} = 2^{-130} \sum_{\substack{0 \leq i \leq 34 \\ i \text{ even}}} \binom{131}{i} \approx 2.472 \times 10^{-8} = 2^{-25.27}, \quad (14)$$

so a trail has mean length $1/\theta_{34} \approx 4.0 \times 10^7$ steps. The same threshold was used by Bailey et al. [1]. A device report includes its seed, trail length, and endpoint coordinates. The host replay starts from the same seed and checks both coordinates and the number of steps. The persisted report is 32 bytes: an eight-byte seed and a 24-byte canonical orbit key.

The benchmark sets the threshold to zero and disables reference replay in the timed loop. Correctness replay is performed separately with a higher reporting threshold. This separation avoids confusing the performance of host reference arithmetic with GPU update throughput.

4 Packed arithmetic and cooperative inversion

4.1 Carryless multiplication and affine updates

An element is held in five 32-bit words, with only three significant bits in the fifth word. A polynomial product has degree at most 260. The selected multiplier uses a 128-bit Karatsuba decomposition: three 64-by-64 carryless products, each supplied by low and high `clmad` instructions, followed by the corrections for the three high input bits and exact modular reduction. NVIDIA specifies `clmad.lo.u64` and `clmad.hi.u64` as carryless multiply-add operations; the PTX instruction was introduced in ISA 9.3 and is available for targets at least `sm_80` [9]. This paper’s measured build targets `sm_120`.

For a selected addend (u, v) , define $d = x + u$ and $e = y + v$. On the ordinary affine domain $d \neq 0$,

$$\lambda = e/d, \quad x' = \lambda^2 + \lambda + d, \quad y' = \lambda(x + x') + x' + y. \quad (15)$$

The implementation retains polynomial state throughout these formulas. In particular, it need not convert both coordinates to normal basis for every addition.

4.2 Weighted prefixes within a thread

Each thread owns $B = 16$ independent scalar slots. For slot i , let

$$D_i = \prod_{j=0}^i d_j, \quad W_i = e_i \prod_{j=0}^{i-1} d_j. \quad (16)$$

The forward pass stores W_i and retains D_{B-1} . Given $v = D_{B-1}^{-1}$, a reverse pass recovers $\lambda_i = vW_i$ and updates $v \leftarrow vd_i$ before proceeding to the previous slot. The denominator is reconstructed from the stored tag and the original x_i , avoiding a separate denominator field in global memory.

The forward pass uses $2(B - 1)$ multiplications. The reverse pass uses B slope products, $B - 1$ inverse-chain products, and B products for the y update. Consequently the ordinary update work, excluding the inverse, totals $5B - 3$ multiplications. The root inverse follows an Itoh–Tsuji chain using eight multiplications: the exponents represented by $\beta_j = a^{2^j - 1}$ progress through $j = 2, 4, 8, 16, 32, 64, 65, 130$, followed by a square. This yields $5B + 5$ multiplications per isolated thread batch.

Algorithm 1 One scheduled batch step, ordinary nonzero-denominator domain

Require: Slots $(x_i, y_i, \mathcal{H}_i)$, $0 \leq i < B$; fixed table; W -warp group

Ensure: Updated coordinates and history; any endpoint reports buffered

```

1:  $D \leftarrow 1$ 
2: for  $i = 0, \dots, B - 1$  do
3:   Evaluate weight and endpoint predicate; update report/dead metadata
4:    $t_i \leftarrow \text{SELECTTAG}(x_i, y_i, \mathcal{H}_i)$ ; push  $t_i$  into  $\mathcal{H}_i$ 
5:    $(u_i, v_i) \leftarrow \text{TABLEADDEND}(t_i)$ ;  $d_i \leftarrow x_i + u_i$ ;  $e_i \leftarrow y_i + v_i$ 
6:    $W_i \leftarrow e_i D$ ;  $D \leftarrow D d_i$  ▷ Omit products by 1 at  $i = 0$ 
7: end for
8:  $v \leftarrow \text{GROUPINVERSE}(D, W)$  ▷ One independent tree per lane
9: for  $i = B - 1, \dots, 0$  do
10:  Reconstruct  $d_i$  from  $t_i$  and original  $x_i$ 
11:   $\lambda \leftarrow v W_i$ ; if  $i > 0$ , set  $v \leftarrow v d_i$ 
12:   $x'_i \leftarrow \text{SQUAREREDUCE}(\lambda) + \lambda + d_i$ 
13:   $y'_i \leftarrow \lambda(x_i + x'_i) + x'_i + y_i$ 
14:  Store  $(x'_i, y'_i)$ 
15: end for

```

4.3 Four-warp product trees

For each lane position, four warps contribute their final batch products to a shared-memory product tree. Combining W products and distributing the inverse costs $3(W - 1)$ multiplications and one root inversion. The arithmetic budget across W batches is therefore

$$W(5B - 3) + 3(W - 1) + 8 = 5BW + 5, \quad (17)$$

or

$$5 + \frac{5}{BW} \quad \text{field multiplications per update.} \quad (18)$$

For $B = 16$, changing $W = 1$ to $W = 4$ reduces this count from 5.3125 to 5.078125, a 4.41% reduction. This is a field-operation count, not a throughput prediction: it excludes barriers, data movement, squarings, table selection, and register effects.

The implementation replaces zero input *batch products* with one before forming the cross-warp tree, then returns zero for those original inputs. This prevents a zero in one thread's batch product from contaminating other threads' inverse results. It does not repair an exceptional affine addition inside the affected thread's batch.

The accepted helper uses a separate named barrier for each group of four complete warps. A 640-thread block contains five such groups. Within each group, logical warp indices are rotated by the group index; the physical roots become warps 3, 6, 9, 12, and 19. This permutation preserves the field results and redistributes the root work. A claimed mapping from those warp identifiers to undocumented physical scheduler assignments is unnecessary and is not assumed.

For $W = 4$, the helper uses seven group barriers per call, including a final barrier that protects scratch reuse. The scratch occupies $5 \cdot 640 \cdot 4 = 12,800$ bytes per block. Partial blocks bypass the collective path and use an individual inverse; the collective barriers are entered only by full groups.

4.4 Square reduction in compressed coefficient streams

In characteristic two, squaring has no cross terms. Define

$$\text{spread} \left(\sum_i a_i 2^i \right) = \sum_i a_i 2^{2i}. \quad (19)$$

The unreduced square of a polynomial bit vector a is $\text{spread}(a)$. A general product reducer accepts up to 261 bits, but a square has zero odd coefficients. The accepted circuit performs the reduction in compressed coefficient streams and interleaves only the final 131-bit answer.

All additions in the following expressions are XOR; shifts operate on nonnegative bit vectors. Set

$$b = a \gg 66, \quad (20)$$

$$E = b \oplus (b \gg 4) \oplus (b \gg 12) \oplus (b \gg 28) \oplus (b \gg 60), \quad (21)$$

$$O = b \oplus (E \gg 1) \oplus (E \gg 2), \quad (22)$$

$$T_E = E \oplus (E \ll 1) \oplus (O \ll 2), \quad (23)$$

$$T_O = O \oplus (O \ll 1) \oplus (E \ll 1). \quad (24)$$

With $S = \{0, 32, 48, 56, 60, 64\}$, compute

$$V_E = \left[a \oplus (E \ll 62) \oplus \bigoplus_{s \in S} (T_E \ll s) \right]_{66}, \quad (25)$$

$$V_O = \left[(O \ll 62) \oplus \bigoplus_{s \in S} (T_O \ll s) \right]_{65}, \quad (26)$$

$$\text{SquareReduce}(a) = \text{spread}(V_E) \oplus (\text{spread}(V_O) \ll 1), \quad (27)$$

where $[\cdot]_r$ truncates to the low r bits. The generated C++ implements these expressions in 32-bit words. Its final zip uses masked exchanges and byte permutation to interleave the even and odd streams; it does not materialize the complete unreduced square.

Proposition 2 (Exactness of the generated square map). *The bit-vector map in Equation (27) equals reduction of $\text{spread}(a)$ modulo Equation (4) for every 131-bit input a .*

Computer-assisted linear-map proof. Both maps are linear over $\mathbb{F}_2$: they consist of coefficient spreading, fixed shifts, truncation, XOR, and reduction by a fixed polynomial. It suffices to compare them on the 131 unit vectors. The supplied generator performs that complete basis comparison against independent polynomial long division. Its check additionally covers zero, all ones, and 10,000 deterministic random values. The manuscript’s local verification reran these checks and regenerated the exact archived header byte for byte. The basis comparison proves the algebraic identity of the two bit-vector maps; the generated C++ and GPU execution are supported separately by implementation probes. $\square$

The local analysis also checks that the internal modulus equals the polynomial reconstructed by the general reducer and passes the binary irreducibility criterion for prime degree 131. These local checks are new analyses of the archived artifact; they are not new GPU timing measurements.

5 GPU storage and scheduling

The accepted persistent field layout stores 256 aligned 16-byte low-part records followed by 256 one-byte tails. Each field tile occupies 4,352 bytes, equivalent to 17 bytes per field element.

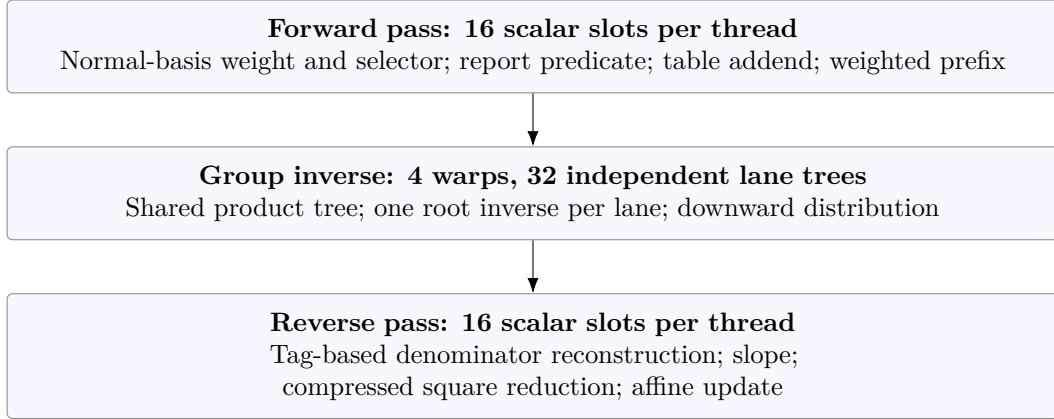


Figure 1: One step of the accepted two-pass kernel. Five four-warp groups fit in each 640-thread block. A lane tree shares an arithmetic inverse; it does not add independent walk points together.

Coordinates and weighted prefixes use three such field arrays, for 51 bytes per scalar slot before metadata. The baseline’s 1,540,096 slots therefore require 78,544,896 field bytes; the candidate’s 1,925,120 slots require 98,181,120 bytes.

Live metadata uses one byte per dead flag and three coalesced planes of 16-bit history tags. The unused fourth history tag is canonicalized when needed. The accepted byte-flag configuration is distinct from earlier experimental bit-packed flags. Checkpoint transfer hooks retain the archived external representation, which is why cross-build continuation is a useful compatibility test.

The kernel receives disjoint coordinate, prefix, metadata, constant, and report buffers through explicit restricted pointers. The underlying allocations justify the non-aliasing contract. The purpose is to permit the compiler to retain and move loads across stores when safe; the annotation is not an arithmetic optimization in itself. The accepted configuration disables additional forward selection pipelining and paired-product instruction interleaving, while retaining paired products and inlined polynomial helpers. The empirical choice reflects the balance between available work and live register pressure.

The receipt reports 94 registers per candidate thread and zero local bytes, compared with 116 registers and zero local bytes for the baseline. Both use 48,732 bytes of dynamic table storage. The candidate adds 12,800 bytes of static inverse scratch, with 1,024 bytes of driver-reserved shared storage. Its total is thus 62,556 bytes per block including the reservation. One block is resident per SM: 20 warps for the candidate and 16 for the baseline. These resource observations explain feasible occupancy; they do not alone establish the mechanism responsible for the measured gain.

The measured device has 188 SMs, 128 MiB of L2, an 80 MiB persisting-L2 allowance, 100 KiB of shared memory per SM, and 65,536 registers per SM. The candidate’s field window exceeds the persisting allowance; the reported requested persistence covers 83,886,080 of its 98,181,120 field bytes. No assumption that all state remains in a particular cache is needed for the reported timing.

6 Experimental methodology

6.1 Artifact and environment

The primary evidence is the completed September 21, 2026 acceptance receipt and exported source tree [6]. The GPU is an NVIDIA RTX PRO 6000 Blackwell Server Edition, driver

580.95.05, with a configured 600 W power limit. The compiler identifies itself as CUDA 13.3, v13.3.73. The archive contains compiler output, device identity, binary hashes, commands, full process output, probes, and the individual timing samples.

The manuscript verification recomputed the measurement hash and checked all 119 entries of the archived source manifest. Every entry matched. An imported repository HEAD is not used as a substitute for those source hashes: the source was imported from a working tree, and the current live configuration can change independently of the archived candidate.

6.2 Meaning of an update and the timing boundary

The program’s completed-run count is

$$N = (I_{\text{end}} - I_{\text{start}}) T B, \quad (28)$$

where T is the number of worker threads and B the number of scalar slots per worker. The packed backend has one scalar point per slot; there is no extra factor of 32 from bitslicing. With 1,024 steps per launch and 256 launches,

$$N_{\text{base}} = 96,256 \cdot 16 \cdot 1,024 \cdot 256 = 403,726,925,824, \quad (29)$$

$$N_{\text{cand}} = 120,320 \cdot 16 \cdot 1,024 \cdot 256 = 504,658,657,280. \quad (30)$$

We call these *scheduled scalar-slot updates*. Every counted slot runs the complete update arithmetic, including selection and coordinate stores. However, the kernel continues to compute a slot after marking its trail finished until the launch ends and reseeding occurs. Such work is included in Equation (28). Its size is small at production settings: a slot finishes with probability θ_{34} per step (Equation (14)) and then idles for half a launch on average, so with 1,024 steps per launch the expected fraction of such updates is about $\theta_{34} \cdot 512 \approx 1.3 \times 10^{-5}$. In benchmark mode the threshold is zero and no slot finishes. Nevertheless a scheduled update must not be interpreted as a newly useful trail point, a distinct visited state, a field multiplication, or a complete scalar multiplication $[k]P$.

The host timer starts after initialization and any checkpoint restore. It covers the launch loop, report fetching and processing, and any reseeding, and ends after a final device synchronization. Thus these are completed client-loop rates, not CUDA-event timings of an isolated kernel. Collection timing includes host report handling and file writes inside that loop. The final file close is after the printed timing boundary. Startup, table construction, initial state generation, compilation, cloud uploads, database ingestion, and an entire collision-resolution campaign are not measured by this metric.

6.3 Confirmation protocol

Each variant receives an excluded full-workload warmup. The harness runs five confirmation rounds and reverses the order of baseline and candidate in alternating rounds. Both variants use the same GPU allocation, 1,024 steps per launch, 256 launches, and their respective normal worker geometry. A valid sample must exit successfully, supply a final completed-run rate, match the expected update count, and report zero dropped endpoints. The paper reports the median and range of the five samples.

These timings compare complete configurations, not individual instructions. Because the candidate has 25% more slots, the samples have different total update counts and durations. Dividing each count by its own measured duration makes a throughput comparison meaningful, while limiting attribution of the gain to any one component. Within the session the ordering is unambiguous: all five candidate samples exceed all five baseline samples, which under the null hypothesis of exchangeable samples has exact one-sided probability $1/\binom{10}{5} = 1/252 \approx$

Round	Baseline, billion/s	Candidate, billion/s
1	20.001884	22.166455
2	19.867106	22.132776
3	19.852741	22.100934
4	19.841065	22.093414
5	19.846254	22.089525
Median	19.852741	22.100934
Minimum	19.841065	22.089525
Maximum	20.001884	22.166455

Table 2: All accepted timing samples in recorded round order; one excluded warmup per variant. Every baseline sample performs 403,726,925,824 scheduled updates and every candidate sample performs 504,658,657,280.

0.004 (Wilcoxon rank-sum). That statement concerns this session only. Five repetitions from one device session are insufficient to estimate variation across machines, drivers, days, or production workloads, and we do not report a population confidence interval.

Higher short-screening results were deliberately excluded from the headline. In particular, the 22.610829- and 22.633932-billion-update/s three-sample screens in the research notebook were used to select a candidate; the new five-sample confirmation determines the reported result. The simpler plain square-reduction candidate was selected over the combined variant, whose small screening advantage was not established independently.

7 Results

7.1 Five-sample confirmation

The ratio of medians is

$$\frac{22.100934}{19.852741} = 1.11324346, \quad (31)$$

corresponding to an 11.3243% improvement. Each candidate run lasts approximately 22.8 seconds under the recorded counting convention. All five candidate rates exceed 22 billion updates/s. The time series is shown in Figure 2; the small downward trend is retained rather than hidden by selecting the fastest sample.

Post-run telemetry reports candidate SM clocks of 2,355–2,362 MHz, temperatures of 64–75 degrees Celsius, and instantaneous power readings of 544.64–584.69 W. The baseline samples report 2,400–2,415 MHz and 60–76 degrees Celsius. The candidate therefore ran about 1.6% slower in clock terms: its median reported SM clock is 2,362 MHz against 2,400 MHz for the baseline, with both variants drawing close to the 600 W limit. Dividing each sample’s rate by its own reported clock gives a ratio of medians of 1.134, a 13.4% gain per clock cycle compared with 11.32% in wall-clock terms. Equivalently, the candidate spends about 20.1 SM clock cycles per update and the baseline about 22.7. The wall-clock figure remains the headline because it is what a user of this power-limited device obtains. The normalized figure is indicative only: these snapshots are taken after each run and are not time-integrated traces. They do not support a joules-per-update or energy-efficiency result, and the frequencies were not fixed across the experiment. Runs with locked clocks, and runs long enough to reach thermal equilibrium, were not performed; each timed run lasts under 23 seconds while temperatures were still rising.

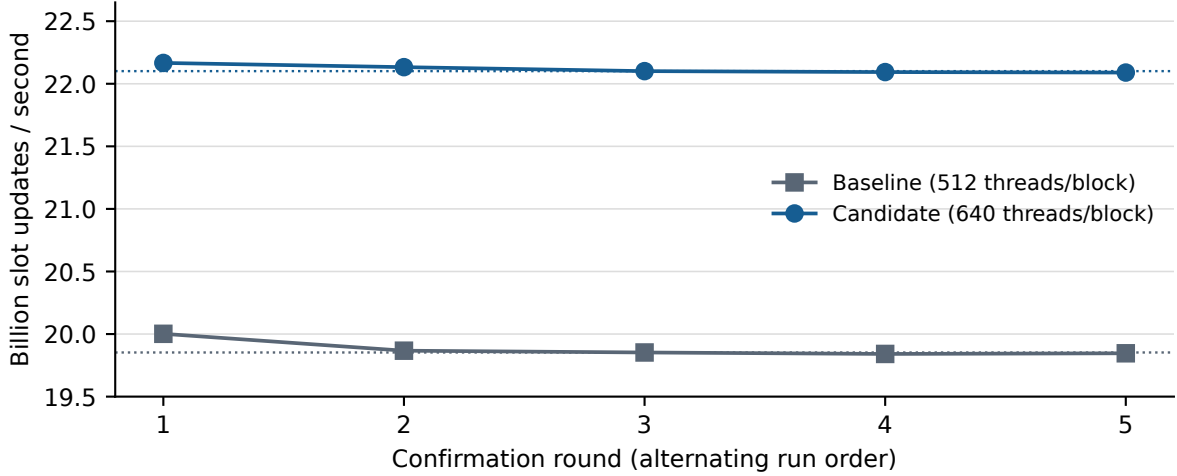


Figure 2: Accepted confirmation rates. Dotted lines show each variant’s median. The vertical axis is deliberately restricted to show within-session drift; numerical values and full counts are provided in Table 2.

7.2 Distinguished-point collection

The separate DP34 candidate run completes the same 504,658,657,280 scheduled updates at 21.548499 billion/s and writes 12,387 records, totaling 396,384 bytes. It reports zero drops. Its rate is 97.5004% of the candidate benchmark median, a descriptive difference of approximately 2.50%. Because the collection result is one sample, this ratio is not a repeated estimate of collection overhead.

For correctness, the baseline collection run is forced to the same 120,320 workers and starting population. The two complete sorted report multisets have the same digest. This baseline geometry is deliberately different from its normal 96,256-worker benchmark geometry, so its 12.288201-billion-update/s collection rate is not used as a speedup denominator. Corpus equality requires equal initial populations; a fair throughput comparison requires appropriately specified configurations. These are different experimental requirements.

7.3 What the measured gain does and does not isolate

The baseline is the unmodified archived source tree, and the candidate is that tree with one patch. Table 3 lists every axis on which the two measured builds differ; all other build switches, including the native carryless multiplier, the weighted prefixes, and the batch size, are identical.

The confirmation compares only the two ends of this list, so it cannot attribute the gain. The multiplication-count reduction in Equation (18) is 4.41%, smaller than the measured 11.32%. The screening receipts give a partial attribution, because the candidate was built up in stages and every stage was timed against a baseline in the same session (Table 4).

Three observations follow. First, the inversion tree at the wider geometry is the largest step, about six percentage points. It depends on the geometry and on root placement: the same tree gave +3.67% at 512 threads, and an earlier unrotated four-warp tree on the unmodified 512-thread kernel was *slower* than its baseline by 3.06%. Second, restricted pointers help only together with the wider block; at 512 threads with interleaving enabled they measured −0.30%. Third, the compressed square adds three to four percentage points over the preceding configurations, and about two over the shorter spread alone. This step is larger than an operation count suggests. The root inverse converts to the normal basis, where squaring is a permutation, so the polynomial square runs only once per update, beside roughly five

Axis	Baseline	Candidate
Inversion	One Itoh–Tsujii inverse per thread ($W = 1$)	Four-warp product tree with rotated roots ($W = 4$)
Polynomial square	Spread to 261 bits, then the general reducer	Compressed even/odd reduction, Section 4.4
Threads per block; warps per SM	512; 16	640; 20
Scalar slots on the device	1,540,096	1,925,120
Kernel buffer arguments	Parameter structure	Restricted pointers
Live metadata per slot	32-bit finished flag; 64-bit history	One byte; three 16-bit tags
Paired-product instruction interleaving	Enabled	Disabled
Registers per thread	116	94
Shared inverse scratch per block	None	12,800 bytes
Field bytes; persisting-L2 coverage	78,544,896; all	98,181,120; 83,886,080

Table 3: Every difference between the two measured builds, taken from the archived patch and from the configuration banner that each run prints. Forward selection pipelining is disabled in both.

Configuration (cumulative)	Arm, billion/s	Baseline, billion/s	Gain
Compact metadata, 640 threads, no interleaving	20.231	19.980	+1.26%
plus restricted pointers	20.564	20.095	+2.33%
plus rotated four-warp inversion	21.756	20.087	+8.31%
plus shorter spread before the general reducer	21.841	19.791	+10.36%
with compressed square reduction instead	22.611	20.071	+12.66%
Same configuration, five-sample confirmation	22.101	19.853	+11.32%

Table 4: Screening sequence. Each row is the median of three samples against the median of three baseline samples from the same session; the last row repeats Table 2. Sessions differ in clocks and temperature, so gains are comparable across rows only approximately.

multiplications. The shorter routine probably also relieves register or scheduling pressure in the reverse pass; an isolated microbenchmark would be needed to confirm that.

This sequence is not a controlled factorial ablation. The stages were measured in different sessions, with three samples each, and no stage removes a single component from the final candidate. A controlled study would disable one row of Table 3 at a time within one alternating session, and would add an isolated microbenchmark of the two squaring routines. That study has not been run. The notebook also records unsuccessful alternatives, including binary-GCD inversion, larger per-thread batches, wider blocks, alternate shared-table layouts, and earlier collective schemes. An earlier primitive-probe include-path error was subsequently corrected; only the final receipt’s intended-header probes and confirmation samples are used for acceptance here.

Check	Recorded coverage	Outcome
Source integrity	119 archived files; measurement digest	All matched locally
Polynomial square identity	131 basis inputs, two edge cases, 10,000 random values	Passed locally
GPU field probes	1,133 squares, 1,133 inverses, 6,798 Frobenius maps	Receipt passed
GPU group inverse	1,024 inputs for each $W = 2, 4, 8, 16$; zeros and consecutive calls	Receipt passed
GPU metadata	1,025 lanes; byte flags; 200 history updates/lane	Receipt passed
Independent trail replay	300 reports with endpoint and length equality	Receipt passed
Complete replay corpus	32,688 records at matched population	Digests equal
DP34 collection corpus	12,387 records; 396,384 bytes	Digests equal
Checkpoint continuation	Baseline $\rightarrow$ candidate and reverse	Receipt passed
Compute Sanitizer	Device/runtime support unavailable	Not passed
Inherited <code>test-clmad</code>	Source/guard expectation failures remain	Not fully green

Table 5: Evidence classes. “Receipt passed” denotes a recorded GPU check, not a new run performed while preparing this manuscript. The algebraic identity check covers the full linear map; the other finite checks have their stated coverage.

8 Correctness evidence and practical limits

8.1 Evidence retained with the result

The independent field probes compare against `Ref<CfgF131>`, rather than merely comparing the candidate with a second use of the same optimized routine. The final receipt also includes the selected collective helper hash and probes the composed arithmetic implementation. The full replay comparison uses 65,536 starting scalar walks, six launches, and 16 steps per launch with reporting threshold 50. Equality is established for the complete sorted multiset of 32,688 persisted records, not just a sampled prefix.

Checkpoint tests split the same workload into three launches in one build and three in the other, then compare against uninterrupted execution. This tests observable continuation across the two memory representations. It does not establish compatibility with every future walk, campaign, or checkpoint format.

The final collection corpora themselves are summarized by retained hashes and counts in the receipt; the manuscript package does not claim to include the original binary corpus files. Reproducing those digests requires rerunning the documented client with the matching seeds and population. Similarly, the measured executable is identified by hash but is not claimed to have been reconstructed byte for byte by the local manuscript build.

8.2 Exceptional affine additions

The fast path implements Equation (15) without complete handling of $d = 0$, and the host table reference intentionally uses the corresponding raw addition. Agreement between those two paths therefore does not establish correct group behavior for an exceptional input. A zero denominator invalidates the affected thread’s local prefix chain: the batch product is zero, the returned inverse is zero, and all 16 slots of that thread receive $\lambda = 0$. Cross-warp zero isolation protects other batches but does not resolve that case.

The exceptional set is small and explicit. The denominator vanishes exactly when the current point and the selected addend share an x coordinate, that is when $R = \pm\sigma^k(T_h)$ for the selected (h, k) . At most $2 \cdot 8 \cdot 131 = 2,096$ subgroup points have this form. Under the usual heuristic that visited points are uniform in the subgroup, the probability per step is at

most

$$\frac{2,096}{\ell} \approx 3.1 \times 10^{-36}, \quad (32)$$

and the expected number of occurrences over an entire search of $2^{60.8}$ steps is about 6×10^{-18} . This bound is heuristic. It assumes that the public table seed does not correlate the table orbits with the starting points, and it is not a proven exclusion condition.

Detection needs no additional field arithmetic. The per-thread batch product is zero exactly when some $d_i = 0$ in that batch, and the collective helper already evaluates that zero test once per thread and step in order to isolate other warps. A complete policy is therefore to mark all 16 slots of a thread as finished whenever its returned inverse is zero, so that the ordinary reseed path replaces them. Discarding 16 trails with probability near 10^{-35} per step has no measurable effect on useful work. This policy is not implemented in the archived candidate and its cost was not timed; the archived timing and replay evidence establish the tested ordinary-domain behavior only.

8.3 Collision quality and useful work

The familiar square-root collision estimate requires an appropriate model of the actual transition system. The estimates in Section 3.3 bound two of the open effects under a uniform model: about 0.1% of merges fail to coalesce because of the history rule, and on the order of 1% of DP34 trails enter a six-step fruitless cycle, which makes the restart limit a first-order parameter of a production run. Neither estimate has been measured, and the nonuniform branch index and the projection to an orbit key are not covered by them. The acceptance receipt does not test the source’s claim that merged trails resynchronize. Equal deterministic output between two implementations of the same walk is a strong regression check, but it does not validate that walk’s random-map model.

A useful next evaluation would measure, on tractable public instances, the distribution of collision times, the proportion of useful independent collisions, trail coalescence after histories differ, and the fraction of trails that reach the restart limit as a function of that limit. The last two measurements test the two estimates above directly. Such a study must preserve the selector and correction rule or explicitly account for any difference. It would supply information that update throughput alone cannot provide. Consequently we do not convert 22.100934 billion updates/s into a projected challenge completion time or a claimed security-bit reduction.

8.4 Statistical and system limits

The experiment uses one GPU allocation and one toolchain/driver combination. Thermal drift is visible and the device is power-limited, so the two variants ran at different clocks; collection has one repetition; the attribution in Table 4 rests on three-sample screens from separate sessions; no locked-clock run, energy integral, multi-device variance, or long-duration sustained-load distribution is available. Compute Sanitizer was unavailable on the device/runtime, and the inherited `test-clmad` suite has unresolved source-shape and architecture-guard expectations. These facts remain explicit even though the selected independent functional probes passed.

The repository also contains cloud persistence and worker-launch evidence. That evidence has a different scope and is excluded from the headline result. No fleet scaling factor, S3/RDS throughput guarantee, or completed challenge computation follows from this single-device acceptance receipt.

9 Reproducibility and artifact availability

The paper’s evidence boundary is the archived directory

`ecc2k130/research/candidates/goal22/`.

The accompanying manuscript package includes the raw measurement receipt, acceptance record, source manifest, derived statistics, and a script that checks the local archive and regenerates the performance figure. The complete source tree remains in that archived directory. This draft does not assert a publicly accessible archival release or invent a repository commit for uncommitted source.

On a compatible CUDA environment, build the archive’s own target:

```
cd ecc2k130/research/candidates/goal22
make gpu-goal22
./ecc2k130 --curve 131 --packed --bench \
--steps 1024 --launches 256 --verify 0
```

This single invocation is a workload command, not the full acceptance protocol. Reproduction must restore the recorded compiler and GPU target, run the independent probes, exclude warmups, alternate the control and candidate over five rounds, retain final counts, and separately repeat collection. The manifest contains the complete 33-token build command and binary identity. The original confirmation also records the baseline build and its output in the measurement receipt.

Before public submission, an immutable source-and-evidence deposit would make these local artifact references independently accessible. The unresolved exceptional-input and collision-quality questions should accompany that deposit, so that a reader can reproduce both the strengths and the limits of the result.

10 Conclusion

The accepted ECC2K-130 table-walk configuration reaches a five-sample median of 22.100934 billion scheduled scalar-slot updates/s on one RTX PRO 6000 Blackwell GPU, improving on its same-run local baseline by 11.32%. A separate collection run reaches 21.548499 billion updates/s with zero reported drops and a matching complete report corpus. The implementation combines native carryless multiplication, polynomial state, four-warp batch inversion, compact metadata, and a square-and-reduce circuit that exploits the even-coefficient structure of binary squaring.

The most transferable result is the joint treatment of arithmetic and GPU scheduling, together with an explicit link from performance claims to immutable evidence. The screening sequence attributes the largest share of the gain to the rotated inversion tree at the wider geometry and three to four percentage points to the square, without a controlled ablation. The selector is defined on every reachable point, and exceptional additions are both negligible in probability and detectable from the existing zero test. The next step toward a complete state-of-the-art collision-search claim is to implement that policy, to measure coalescence and the six-step cycle rate of the history-dependent walk on tractable instances, and then to measure the resulting useful work with a restart limit under sustained, locked-clock conditions.

References

- [1] Daniel V. Bailey, Lejla Batina, Daniel J. Bernstein, Peter Birkner, Joppe W. Bos, Hsieh-Chung Chen, Chen-Mou Cheng, Gauthier van Damme, Giacomo de Meulenaer, Luis Julian Dominguez Perez, Junfeng Fan, Tim Güneysu, Frank Gürkaynak, Thorsten

- Kleinjung, Tanja Lange, Nele Mentens, Ruben Niederhagen, Christof Paar, Francesco Regazzoni, Peter Schwabe, Leif Uhsadel, Anthony Van Herrewege, and Bo-Yin Yang. Breaking ECC2K-130. Cryptology ePrint Archive, Report 2009/541, 2009. <https://eprint.iacr.org/2009/541>.
- [2] Daniel J. Bernstein, Hsieh-Chung Chen, Chen-Mou Cheng, Tanja Lange, Ruben Niederhagen, Peter Schwabe, and Bo-Yin Yang. ECC2K-130 on NVIDIA GPUs. Cryptology ePrint Archive, Report 2012/002, 2012. Revised manuscript of INDOCRYPT 2010, pp. 328–346. <https://eprint.iacr.org/2012/002>.
- [3] Daniel J. Bernstein and Tanja Lange. Type-II optimal polynomial bases. In *Arithmetic of Finite Fields – WAIFI 2010*, volume 6087 of *Lecture Notes in Computer Science*, pages 41–61. Springer, 2010. <https://eprint.iacr.org/2010/069>.
- [4] Daniel J. Bernstein, Tanja Lange, and Peter Schwabe. On the correct use of the negation map in the Pollard rho method. In *Public Key Cryptography – PKC 2011*, volume 6571 of *Lecture Notes in Computer Science*, pages 128–146. Springer, 2011. <https://cr.yp.to/elliptic/negation-20110102.pdf>.
- [5] Joppe W. Bos, Thorsten Kleinjung, Ruben Niederhagen, and Peter Schwabe. ECC2K-130 on Cell CPUs. In *Progress in Cryptology – AFRICACRYPT 2010*, volume 6055 of *Lecture Notes in Computer Science*, pages 225–242. Springer, 2010. <https://eprint.iacr.org/2010/077>.
- [6] Adam Buran. ECC2K-130 accepted Blackwell candidate: warps4_poly_compact640. Project research artifact, September 21, 2026. Local archive `ecc2k130/research/candidates/goal22/`. Content hashes appear in Appendix A. Receipt, acceptance, and manifest are bundled with this manuscript; public archival deposit is pending.
- [7] Toshiya Itoh and Shigeo Tsujii. A fast algorithm for computing multiplicative inverses in $GF(2^m)$ using normal bases. *Information and Computation*, 78(3):171–177, 1988. [https://doi.org/10.1016/0890-5401\(88\)90024-7](https://doi.org/10.1016/0890-5401(88)90024-7).
- [8] Peter L. Montgomery. Speeding the Pollard and elliptic curve methods of factorization. *Mathematics of Computation*, 48(177):243–264, 1987.
- [9] NVIDIA. Parallel Thread Execution ISA, version 9.4, 2026. Section 9.7.1.5, carryless multiply-add. Accessed September 21, 2026. <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#integer-arithmetic-instructions-clmad>.
- [10] John M. Pollard. Monte carlo methods for index computation (mod p). *Mathematics of Computation*, 32(143):918–924, 1978. <https://doi.org/10.1090/S0025-5718-1978-0491431-9>.
- [11] Paul C. van Oorschot and Michael J. Wiener. Parallel collision search with cryptanalytic applications. *Journal of Cryptology*, 12:1–28, 1999. Author manuscript available at <https://people.scs.carleton.ca/~paulv/papers/JoC97.pdf>.

A Artifact identities and implementation map

Object	SHA-256
Measurement receipt	840b6071d202393a958b00317be58d90f b2a9367363659d18daaaebba2dc4d82
Candidate executable	da24894bdef26c638b243790a3bf4cf1 66feecac27221cd8c035191dc20d9e9a
Baseline executable	b62f1c6162164dcea69c7478525f8a0a f65bb295420ce919da9830f4fc1a0500
Replay corpus (sorted)	db31c5c36206192fe8fda4cba58152e4 6620d2ffe5e4d22605e7f7d34909162e
DP34 corpus (sorted)	334b830fc6d3bf5801013203010bb030 baddfdc5950f11c8a881a5c95ff9c092

The recorded GPU UUID is GPU-080ac0e6-6dbe-3064-5ce3-7fe34abd5193. The archived candidate name, hashes, and source manifest identify this study even if the live build profile changes.

Paper component	Archived implementation
Curve and field constants	<code>generated/eccF131.h</code>
Reference table and raw update	<code>include/ref.h</code> , <code>TableWalk<Cfg></code>
Phase, pivot, cycle tags	<code>include/tablewalk.h</code>
Device selection and addends	<code>include/packedtablewalk.cuh</code>
Algorithm 1 forward/reverse passes	<code>include/packedkernels.cuh</code> , <code>walk</code>
Four-warp inverse	<code>include/collective_inverse.cuh</code> , <code>goal22CollectiveInverse<4></code>
Compressed square reduction	<code>research/gen_square_reduce.py</code> ; <code>include/square_reduce.h</code>
Packed state and metadata	<code>include/packedcompactstate.cuh</code> ; <code>include/compact_metadata.cuh</code>
Timer and update accounting	<code>src/main.cu</code> , completed-run output

Table 6: Mapping between manuscript algorithms and the frozen source. All paths in this table are relative to the archived candidate directory.

B Zero handling and product-tree invariant

For nonzero leaves $d_0, \dots, d_{W-1}$, each upward tree node stores the product of its descendant leaves. If a node with children L, R holds $(LR)^{-1}$, its child inverses are $R(LR)^{-1} = L^{-1}$ and $L(LR)^{-1} = R^{-1}$. The downward pass therefore returns each original leaf inverse. Replacing a zero leaf by one before this construction keeps the product invertible; restoring zero at that output implements the helper’s explicit convention $\text{inv}(0) = 0$.

The logical warp permutation is bijective within each barrier group and leaves lane positions unchanged. Each group consequently runs 32 separate product trees, and no operand crosses a group boundary. This algebraic invariant explains the returned values. Race freedom and barrier participation still rely on the actual synchronization implementation and are supported by finite device tests, with the sanitizer limitation reported in Table 5.